1. Create a tomcat container on 8080 and deploy sample application in tomcat.

1. Objective

The objective of this task was to create and run an Apache Tomcat Docker container on port 8080 and deploy a sample web application inside the container.

The application was tested to verify that Tomcat was running and serving the deployed HTML page through the browser.

2. Purpose

The purpose of this task was to understand how Docker containers can be used to run application servers and host web applications.

- This practical exercise demonstrates:
- Pulling a Tomcat image from Docker Hub.
- Creating and running a Docker container.
- Mapping the EC2 host port to the container port.
- Deploying a sample HTML application inside Tomcat.
- Configuring an AWS EC2 security group to allow application access.
- Testing the application using curl and a web browser.
- Inspecting container status and logs for troubleshooting.

3. Implementation Steps

Step 1: Install and Start Docker
If Docker was not already installed, installed it on Amazon Linux 2023 using:

→ sudo dnf update -y
→ sudo dnf install docker -y
→ Started and enabled the Docker service:
→ sudo systemctl start docker
→ sudo systemctl enable docker

Verified the installation:
→ sudo docker –version
→sudo docker info

Step 2: Download the Tomcat Image
Downloaded the Tomcat image from Docker Hub:
→ sudo docker pull tomcat:10.1-jdk21-temurin
Verified that the image was available locally:
→ sudo docker images

Step 3: Create a Working Directory

Created a directory for the sample application:

→ mkdir -p ~/docker-assignment/task1
→ cd ~/docker-assignment/task1

Step 4: Create the Sample HTML Application

Created an HTML file named index.html:

```
cat > index.html <<'EOF'
<!DOCTYPE html>
<html>
<head>
   <title>Docker Tomcat Task 1</title>
</head>
<body>
   <h1>Tomcat Container is Working!</h1>
   <p>Task 1: Sample application deployed successfully.</p>
   <p>Application Server: Apache Tomcat</p>
   <p>Container Port: 8080</p>
</body>
</html>
EOF
```

Verified the file:

```
cat index.html
```

Step 5: Create the Tomcat Container

Created and started the container:

```
sudo docker run -d \
  --name tomcat-8080 \
  -p 8080:8080 \
  tomcat:10.1-jdk21-temurin
```

Explanation:

docker run: Creates and starts a container.
-d: Runs the container in the background.
--name tomcat-8080: Assigns a name to the container.
-p 8080:8080: Maps host port 8080 to container port 8080.
tomcat:10.1-jdk21-temurin: Specifies the Tomcat image.

Verified the running container:

```
sudo docker ps
```

Step 6: Deploy the Application

Created the Tomcat root application directory:

→ sudo docker exec tomcat-8080 \
mkdir -p /usr/local/tomcat/webapps/ROOT

Copied the sample HTML file into the container:

→ sudo docker cp index.html \
tomcat-8080:/usr/local/tomcat/webapps/ROOT/index.html

Verified the deployed application:

→ sudo docker exec tomcat-8080 \
ls -l /usr/local/tomcat/webapps/ROOT/

Step 7: Test the Application Locally

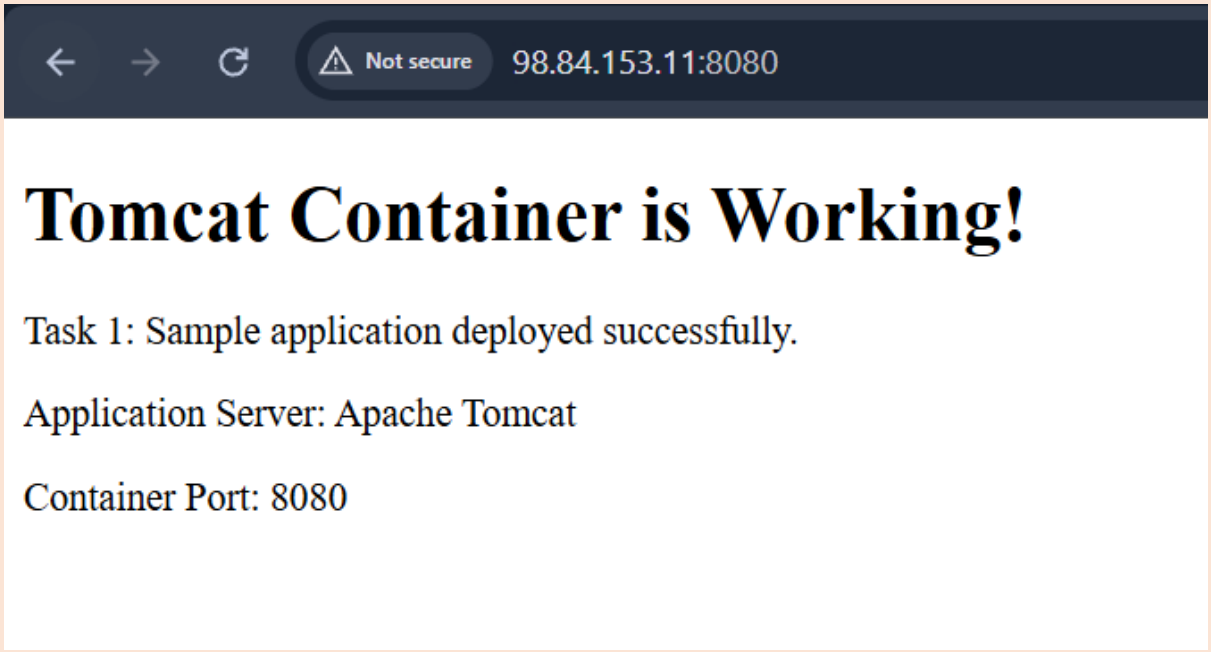
Executed the following command on the EC2 instance: curl -i http://localhost:8080/
Verified that Tomcat returned the application page and an HTTP success response.
Checked the container logs when necessary: sudo docker logs --tail 50 tomcat-8080

Implementation:

```
[root@ip-172-31-27-111 ec2-user]# sudo systemctl start docker
[root@ip-172-31-27-111 ec2-user]# sudo systemctl enable docker
Created symlink /etc/systemd/system/multi-user.target.wants/docker.service → /usr/lib/systemd/system/docker.service.
[root@ip-172-31-27-111 ec2-user]# sudo systemctl status docker --no-pager
● docker.service - Docker Application Container Engine
     Loaded: loaded (/usr/lib/systemd/system/docker.service; enabled; preset: disabled)
     Active: active (running) since Tue 2026-10-13 10:25:02 UTC; 15s ago
TriggeredBy: ● docker.socket
       Docs: https://docs.docker.com
   Main PID: 29656 (dockerd)
      Tasks: 9
     Memory: 27.3M
        CPU: 198ms
     CGroup: /system.slice/docker.service
             └─29656 /usr/bin/dockerd -H fd:// --containerd=/run/containerd/containerd.sock --default-ulimit nofile=32768:65536

Oct 13 10:25:02 ip-172-31-27-111.ec2.internal systemd[1]: Starting docker.service - Docker Application Container Engine...
Oct 13 10:25:02 ip-172-31-27-111.ec2.internal dockerd[29656]: time="2026-10-13T10:25:02.356387791Z" level=info msg="Starting up"
Oct 13 10:25:02 ip-172-31-27-111.ec2.internal dockerd[29656]: time="2026-10-13T10:25:02.387041442Z" level=info msg="Loading containers: start."
Oct 13 10:25:02 ip-172-31-27-111.ec2.internal dockerd[29656]: time="2026-10-13T10:25:02.756712691Z" level=info msg="Loading containers: done."
Oct 13 10:25:02 ip-172-31-27-111.ec2.internal dockerd[29656]: time="2026-10-13T10:25:02.769722557Z" level=info msg="Docker daemon" commit=6fdf0a6 containerd-s
Oct 13 10:25:02 ip-172-31-27-111.ec2.internal dockerd[29656]: time="2026-10-13T10:25:02.769867839Z" level=info msg="Daemon has completed initialization"
Oct 13 10:25:02 ip-172-31-27-111.ec2.internal systemd[1]: Started docker.service - Docker Application Container Engine.
Oct 13 10:25:02 ip-172-31-27-111.ec2.internal dockerd[29656]: time="2026-10-13T10:25:02.789020251Z" level=info msg="API listen on /run/docker.sock"
Hint: Some lines were ellipsized, use -l to show in full.
[root@ip-172-31-27-111 ec2-user]#
```

```
[root@ip-172-31-27-111 task1]# cat > index.html <<'EOF'
<!DOCTYPE html>
<html>
<head>
    <title>Docker Tomcat Task 1</title>
</head>
<body>
    <h1>Tomcat Container is Working!</h1>
    <p>Task 1: Sample application deployed successfully.</p>
    <p>Application Server: Apache Tomcat</p>
    <p>Container Port: 8080</p>
</body>
</html>
EOF
[root@ip-172-31-27-111 task1]# cat index.html
<!DOCTYPE html>
<html>
<head>
    <title>Docker Tomcat Task 1</title>
</head>
<body>
    <h1>Tomcat Container is Working!</h1>
    <p>Task 1: Sample application deployed successfully.</p>
    <p>Application Server: Apache Tomcat</p>
    <p>Container Port: 8080</p>
</body>
</html>
[root@ip-172-31-27-111 task1]# sudo docker ps
CONTAINER ID   IMAGE     COMMAND   CREATED   STATUS    PORTS     NAMES
[root@ip-172-31-27-111 task1]# sudo docker run -d \
  --name tomcat-8080 \
  -p 8080:8080 \
  tomcat:10.1-jdk21-temurin
20633a59bdeae3ae1aac8fabe3830e205a2801c79e78b7cea7f40d0f1a698d9c
```



4. Conclusion

Successfully completed the practical exercise of running an Apache Tomcat Docker container on port 8080 and deploying a sample HTML application.

This task provided practical knowledge of Docker image management, container creation, port mapping, application deployment, AWS security group configuration, and basic application troubleshooting.

The exercise demonstrated how a web application can be hosted inside a Docker container and accessed remotely through an EC2 instance.

2. Create volume and deploy tomcat container on port 8081.

1. Objective

The objective of this task was to create a Docker named volume, mount it inside a Tomcat container, and deploy a sample web application accessible through port 8081 on the Amazon EC2 instance.

The task also demonstrated how application files stored in a Docker volume can persist even when the original container is removed and recreated.

2. Purpose

The purpose of this task was to understand Docker persistent storage and the relationship between containers, volumes, and application data.

This exercise covered:

- Creating and inspecting a Docker named volume.
- Mounting the volume inside a Tomcat container.
- Deploying a sample HTML application.
- Mapping EC2 host port 8081 to container port 8080.

- Configuring the AWS security group for application access.
- Verifying application availability.
- Demonstrating persistence after container recreation.

3. Implementation Steps

Step 1: Verify Docker
Checked Docker installation and running containers:
sudo docker --version
sudo docker ps

Step 2: Create a Docker Volume
Created a named volume:
→ sudo docker volume create tomcat8081-data

Inspected the volume:
→ sudo docker volume inspect tomcat8081-data

Verified the volume:
→ sudo docker volume ls

Step 3: Create the Tomcat Container

Created the container using the Tomcat image:

```
sudo docker run -d \
  --name tomcat-8081 \
  -p 8081:8080 \
  -v tomcat8081-data:/usr/local/tomcat/webapps/ROOT \
  tomcat:10.1-jdk21-temurin
```

The container was configured with the following settings:

Container name: tomcat-8081
Host port: 8081
Container port: 8080
Volume name: tomcat8081-data
Volume mount path: /usr/local/tomcat/webapps/ROOT

Step 4: Deploy the Sample Application

Copied the HTML file created in Task 1 into the container's mounted directory:

```
sudo docker cp \
  ~/docker-assignment/task1/index.html \
  tomcat-8081:/usr/local/tomcat/webapps/ROOT/index.html
```

Verified the deployed file:

```
sudo docker exec tomcat-8081 \
 ls -l /usr/local/tomcat/webapps/ROOT/
```

Opened the application in a browser: http://YOUR_EC2_PUBLIC_IP:8081/

Step 7: Verify Data Persistence

Stopped and removed the original container:

→ sudo docker stop tomcat-8081

→ sudo docker rm tomcat-8081

Verified that the Docker volume still existed:

→ sudo docker volume ls

Recreated the Tomcat container using the same named volume:

→ sudo docker run -d \
--name tomcat-8081 \
-p 8081:8080 \
-v tomcat8081-data:/usr/local/tomcat/webapps/ROOT \
tomcat:10.1-jdk21-temurin

Verified that the HTML file persisted:

→ sudo docker exec tomcat-8081 \
cat /usr/local/tomcat/webapps/ROOT/index.html

Tested the application again:

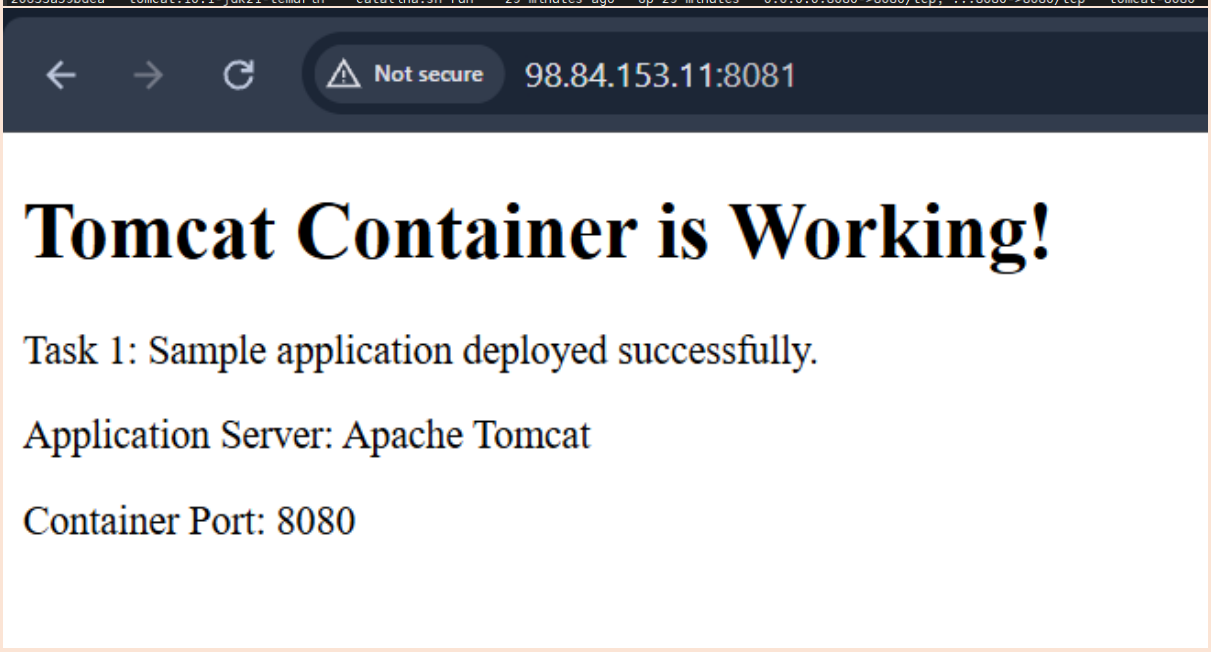
→ curl -i http://localhost:8081/

Implementatin:

```
[root@ip-172-31-27-111 ec2-user]# sudo docker cp \
  ~/docker-assignment/task1/index.html \
  tomcat-8081:/usr/local/tomcat/webapps/ROOT/index.html
Successfully copied 2.05kB to tomcat-8081:/usr/local/tomcat/webapps/ROOT/index.html
[root@ip-172-31-27-111 ec2-user]# sudo docker exec tomcat-8081 \
  ls -l /usr/local/tomcat/webapps/ROOT/
total 4
-rw-r--r--+ 1 root root 281 Oct 13 10:29 index.html
[root@ip-172-31-27-111 ec2-user]# curl -i http://localhost:8081/
HTTP/1.1 200
Accept-Ranges: bytes
ETag: W/"281-1791887363000"
Last-Modified: Tue, 13 Oct 2026 10:29:23 GMT
Content-Type: text/html
Content-Length: 281
Date: Tue, 13 Oct 2026 11:08:37 GMT

<!DOCTYPE html>
<html>
<head>
    <title>Docker Tomcat Task 1</title>
</head>
<body>
    <h1>Tomcat Container is Working!</h1>
    <p>Task 1: Sample application deployed successfully.</p>
    <p>Application Server: Apache Tomcat</p>
    <p>Container Port: 8080</p>
</body>
</html>
```

```
[root@ip-172-31-27-111 ec2-user]# ls -l ~/docker-assignment/task1/index.html
-rw-r--r--. 1 root root 281 Oct 13 10:29 /root/docker-assignment/task1/index.html
[root@ip-172-31-27-111 ec2-user]# sudo docker volume create tomcat8081-data
tomcat8081-data
[root@ip-172-31-27-111 ec2-user]# sudo docker volume inspect tomcat8081-data
[
    {
        "CreatedAt": "2026-10-13T10:57:47Z",
        "Driver": "local",
        "Labels": null,
        "Mountpoint": "/var/lib/docker/volumes/tomcat8081-data/_data",
        "Name": "tomcat8081-data",
        "Options": null,
        "Scope": "local"
    }
]
[root@ip-172-31-27-111 ec2-user]# sudo docker volume ls
DRIVER    VOLUME NAME
local     tomcat8081-data
[root@ip-172-31-27-111 ec2-user]# sudo docker ps
CONTAINER ID   IMAGE                      COMMAND             CREATED          STATUS          PORTS                                       NAMES
20633a59bdea   tomcat:10.1-jdk21-temurin   "catalina.sh run"   28 minutes ago   Up 28 minutes   0.0.0.0:8080->8080/tcp, :::8080->8080/tcp   tomcat-8080
[root@ip-172-31-27-111 ec2-user]# sudo docker run -d \
  --name tomcat-8081 \
  -p 8081:8080 \
  -v tomcat8081-data:/usr/local/tomcat/webapps/ROOT \
  tomcat:10.1-jdk21-temurin
f4036c13de1b574452253db50614ed0b53d5f11953947fd8445108d0f3abf417
[root@ip-172-31-27-111 ec2-user]# sudo docker ps
CONTAINER ID   IMAGE                      COMMAND             CREATED          STATUS          PORTS                                       NAMES
f4036c13de1b   tomcat:10.1-jdk21-temurin   "catalina.sh run"   9 seconds ago    Up 8 seconds    0.0.0.0:8081->8080/tcp, :::8081->8080/tcp   tomcat-8081
20633a59bdea   tomcat:10.1-jdk21-temurin   "catalina.sh run"   29 minutes ago   Up 29 minutes   0.0.0.0:8080->8080/tcp, :::8080->8080/tcp   tomcat-8080
```



4. Conclusion

Successfully completed the exercise of creating a Docker named volume and deploying a Tomcat container on port 8081.

The task demonstrated Docker volume creation, volume mounting, port mapping, application deployment, and persistent storage.

It also established an important Docker concept: removing a container does not automatically remove its named volume. The volume can be reused by another container, allowing application files to persist independently of the original container's lifecycle.

3. Limit the nginx container to 500 MB.

1. Objective

The objective of this task was to create an Nginx Docker container and configure a memory limit of 500 MB using Docker resource-management options.

The container was tested to verify that Nginx was running and that Docker had applied the configured memory limit.

2. Purpose

The purpose of this task was to understand Docker container resource management and memory restrictions.

This exercise covered:

- Downloading the official Nginx image.
- Creating a Docker container with a memory limit.
- Configuring the combined memory-and-swap limit.
- Publishing a container port on the EC2 host.
- Verifying memory settings using Docker inspection commands.
- Monitoring memory usage.
- Testing the Nginx web server.

3. Implementation Steps

Step 1: Check Existing Containers

Executed:

```
sudo docker ps
sudo docker stats --no-stream
```

Step 2: Download the Nginx Image

Downloaded the official Nginx image: `sudo docker pull nginx:latest`
Verified the image: `sudo docker images nginx`

Step 3: Create the Nginx Container with a Memory Limit

Created and started the container:

```
sudo docker run -d \
  --name nginx-500mb \
  -p 8082:80 \
  --memory=500m \
  --memory-swap=500m \
  nginx:latest
```

Configuration details:
Container name: nginx-500mb
Host port: 8082
Container port: 80
Memory limit: 500 MiB
Combined memory-and-swap limit: 500 MiB
Image: nginx:latest

The --memory=500m option configured the container's memory limit. The --memory-swap=500m option set the combined memory-and-swap allowance to the same limit.

Step 4: Verify the Container

Executed: sudo docker ps

Checked the configured memory limit:

```
sudo docker inspect nginx-500mb \
  --format 'Memory={{.HostConfig.Memory}}'
```

Checked the memory-and-swap setting:

```
sudo docker inspect nginx-500mb \
  --format 'MemorySwap={{.HostConfig.MemorySwap}}'
```

The expected value for each setting was: 524288000
This value represents 500 MiB in bytes.

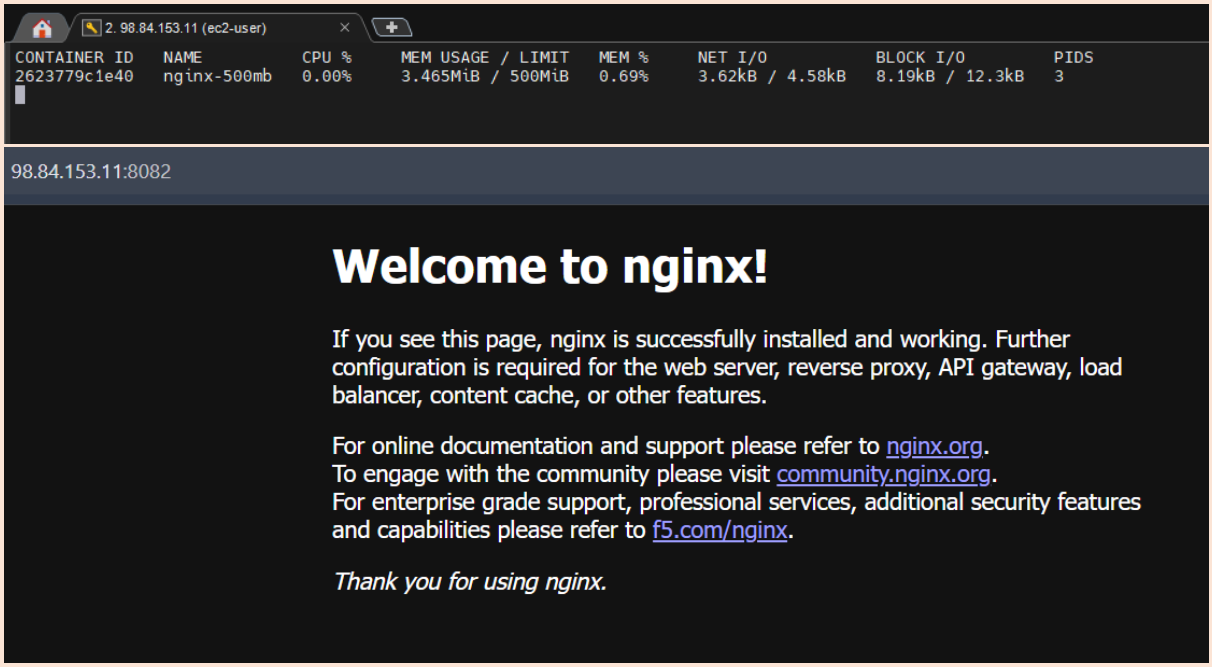
Step 5: Monitor Memory Usage

Executed: sudo docker stats --no-stream nginx-500mb

Step 6: Test Nginx

Tested the application locally: curl -I http://localhost:8082/
Retrieved the default web page: curl http://localhost:8082/

Implementation:



5. Conclusion

Successfully completed the exercise of configuring an Nginx Docker container with a 500 MiB memory limit.

This task demonstrated Docker resource controls, container port mapping, memory monitoring, container inspection, and application verification.

The exercise highlighted the importance of setting appropriate resource limits to help manage container workloads on shared infrastructure.

4. Create a sample docker file using below instructions.

a) Base module as amazonlinux:latest
b) Maintainer you name
c) Install nginx
d) COPY one index.html file to image
e) Expose on port 80
f) Command to start the nginx container

1. Objective

The objective of this task was to create a custom Docker image using amazonlinux:latest as the base image, install Nginx, copy a sample HTML file into the image, expose port 80, and configure Nginx to start automatically when a container is launched.

2. Purpose

The purpose of this task was to understand how Dockerfiles are used to automate image creation and package an application with its required runtime components.

This exercise covered:

- Writing a Dockerfile.
- Selecting a base image.
- Installing Nginx during the image build.
- Copying application files into the image.
- Defining the exposed container port.
- Configuring the container's startup command.
- Building and inspecting a custom Docker image.
- Running and testing a container from the image.

3. Implementation Steps

Step 1: Create a Working Directory

Created a dedicated directory for the task:

```
mkdir -p ~/docker-assignment/task4
cd ~/docker-assignment/task4
```

Step 2: Create the Sample HTML File

Created index.html:

```
cat > index.html <<'EOF'
<!DOCTYPE html>
<html>
<head>
   <title>Custom Amazon Linux Nginx Image</title>
</head>
<body>
   <h1>Custom Docker Image Working Successfully!</h1>
   <p>Task 4: Dockerfile implementation completed.</p>
   <p>Base Image: Amazon Linux</p>
   <p>Web Server: Nginx</p>
   <p>Container Port: 80</p>
</body>
</html>
EOF
```

Step 3: Create the Dockerfile

Created a file named Dockerfile:

```
FROM amazonlinux:latest
MAINTAINER Sunith
RUN dnf install -y nginx && dnf clean all
COPY index.html /usr/share/nginx/html/index.html
EXPOSE 80
CMD ["nginx", "-g", "daemon off;"]
```

The maintainer placeholder was intended to be replaced with the actual maintainer name.

Step 4: Build the Custom Image

Built the Docker image: `sudo docker build -t custom-nginx-amazonlinux:1.0 .`

The build process used Amazon Linux as the base image, installed Nginx, copied the HTML file, and saved the resulting image locally.

Step 5: Verify the Image

Executed: sudo docker images

Confirmed that the image was available with the repository name custom-nginx-amazonlinux and tag 1.0.

Inspected the image configuration: `sudo docker image inspect custom-nginx-amazonlinux:1.0`

Verified the exposed port:

```
sudo docker image inspect \
  custom-nginx-amazonlinux:1.0 \
  --format '{{json .Config.ExposedPorts}}'
```

Expected output: `{"80/tcp":{}}`

Step 6: Create a Container from the Image

Started the custom Nginx container:

```
sudo docker run -d \
  --name custom-nginx-task4 \
  -p 8083:80 \
  custom-nginx-amazonlinux:1.0
```

Step 7: Verify the Container and Application

Checked the container: sudo docker ps
Checked the logs: sudo docker logs custom-nginx-task4
Tested the application: curl -i http://localhost:8083/

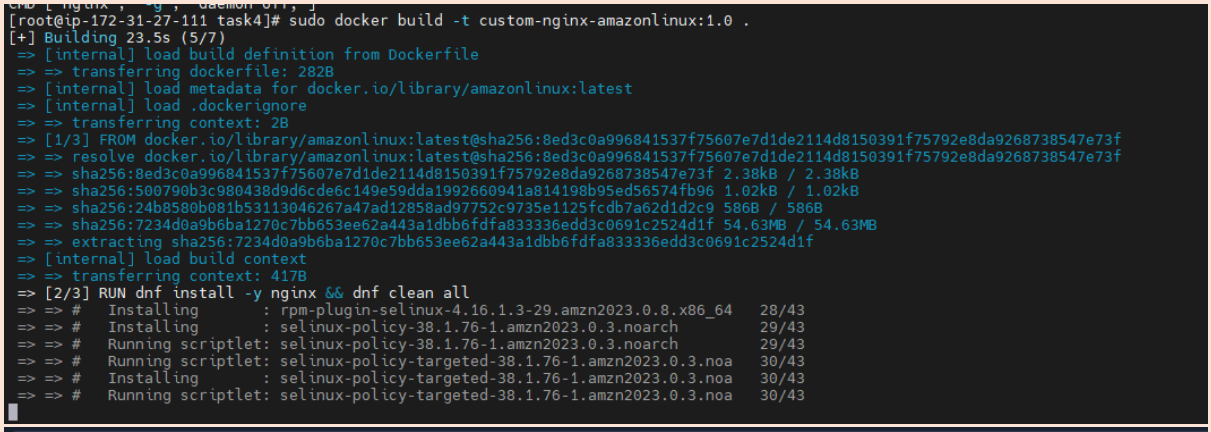
The expected result was an HTTP success response containing the custom HTML page.

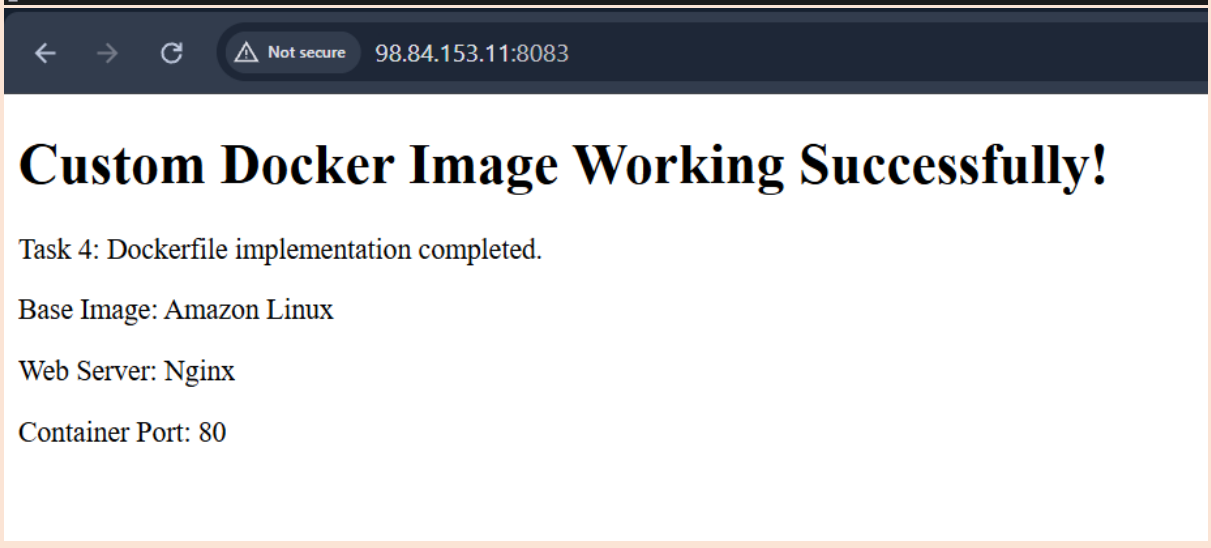
Step 8: Configure Browser Access

Opened the application using: http://YOUR_EC2_PUBLIC_IP:8083/

Replaced the placeholder with the EC2 instance's public IPv4 address.

Implementation:





5. Conclusion

Successfully completed the exercise of creating a custom Docker image using Amazon Linux and Nginx.

The task demonstrated how Dockerfiles automate the installation of software, inclusion of application content, port documentation, and container startup configuration.

The resulting custom image can be reused to create additional containers and distributed to image registries such as Docker Hub and Amazon Elastic Container Registry.

5. Push image to Docker hub

1. Objective

The objective of this task was to publish the custom Docker image created in Task 4 to Docker Hub so that it could be stored in a remote image repository and reused on other machines.

2. Purpose

The purpose of this task was to understand how Docker images are tagged, authenticated, uploaded, and retrieved from a container registry.

This exercise covered:

- Creating a Docker Hub repository.
- Authenticating Docker with Docker Hub.
- Tagging a local image with a registry repository name.
- Pushing an image to Docker Hub.
- Verifying the image and tag in the remote repository.
- Pulling the published image for reuse.

3. Implementation Steps

Step 1: Create a Docker Hub Repository

Signed in to Docker Hub and created a repository named: custom-nginx-amazonlinux

Selected the required visibility setting and confirmed that the repository was available.

Step 2: Verify the Local Image

Executed: sudo docker images

Confirmed that the custom image created in Task 4 was available locally:
custom-nginx-amazonlinux:1.0

Step 3: Authenticate with Docker Hub

Logged in using the Docker Hub username: sudo docker login
--username and –Password
Used the appropriate Docker Hub credentials or personal access token when prompted.
Verified that authentication succeeded.

Step 4: Tag the Image

Tagged the local image with the Docker Hub username and repository name:

```
sudo docker tag \
  custom-nginx-amazonlinux:1.0 \
  sunith18/custom-nginx-amazonlinux:1.0
```

Replaced the placeholder with the actual Docker Hub username.

Step 5: Push the Image

Uploaded the image to Docker Hub:

```
sudo docker push \
  sunith18//custom-nginx-amazonlinux:1.0
```

Waited for Docker to finish uploading the image layers.

Step 7: Pull the Published Image

Tested retrieval of the published image:

```
sudo docker pull \
  sunith18/custom-nginx-amazonlinux:1.0
```

Verified the downloaded image: sudo docker images

Step 8: Test the Retrieved Image

Created a test container from the published image:

```
sudo docker run -d \
  --name custom-nginx-dockerhub-test \
  -p 8084:80 \
  Sunith18/custom-nginx-amazonlinux:1.0
```

Tested the application: curl -i http://localhost:8084/

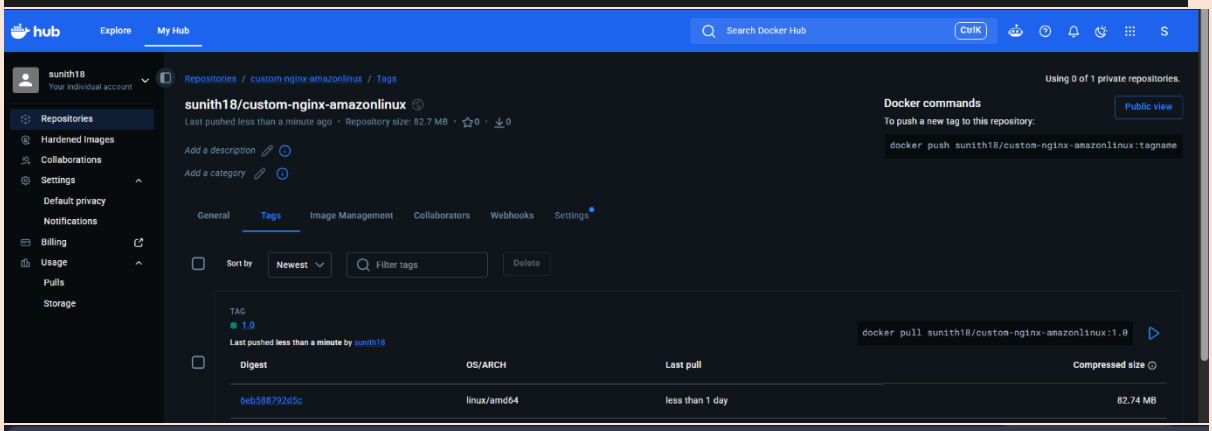
The expected result was an HTTP success response containing the custom HTML page.

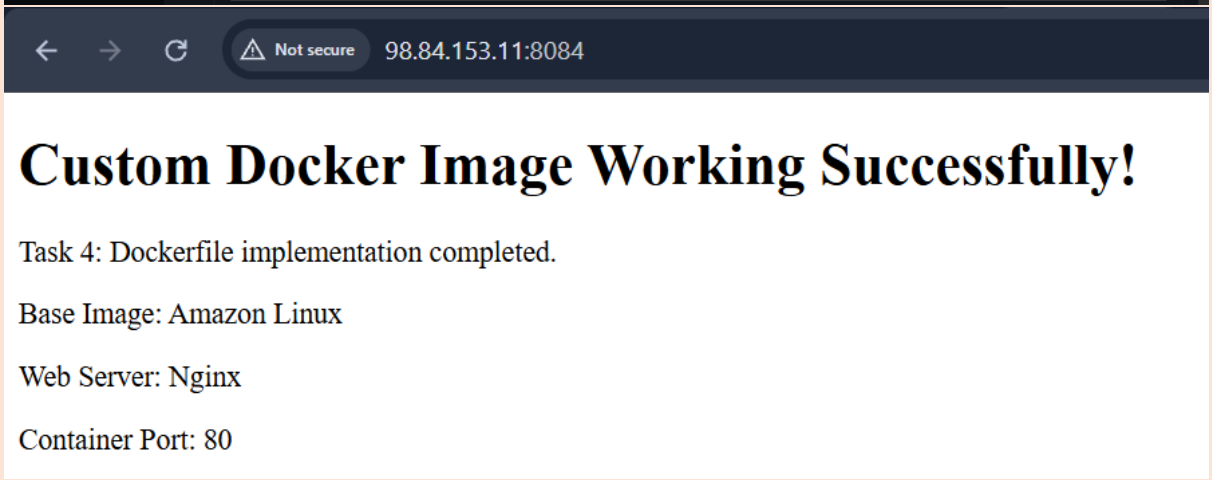
Implementation:

```
[root@ip-172-31-27-111 task4]# docker login
Log in with your Docker ID or email address to push and pull images from Docker Hub. If you don't have a Docker ID, he
You can log in with your password or a Personal Access Token (PAT). Using a limited-scope PAT grants better security a
kens/

Username: sunith18
Password:
WARNING! Your password will be stored unencrypted in /root/.docker/config.json.
Configure a credential helper to remove this warning. See
https://docs.docker.com/engine/reference/commandline/login/#credentials-store

Login Succeeded
[root@ip-172-31-27-111 task4]# sudo docker tag \
  custom-nginx-amazonlinux:1.0 \
  sunith18/custom-nginx-amazonlinux:1.0
[root@ip-172-31-27-111 task4]# sudo docker images
REPOSITORY                           TAG                 IMAGE ID       CREATED          SIZE
custom-nginx-amazonlinux             1.0                 cb819c3670f1   22 minutes ago   235MB
sunith18/custom-nginx-amazonlinux    1.0                 cb819c3670f1   22 minutes ago   235MB
tomcat                               10.1-jdk21-temurin  d654718a0188   13 hours ago     505MB
nginx                                latest              57df7715e516   7 days ago       163MB
hello-world                          latest              e2ac70e7319a   6 months ago     10.1kB
[root@ip-172-31-27-111 task4]# sudo docker push \
  sunith18/custom-nginx-amazonlinux:1.0
The push refers to repository [docker.io/sunith18/custom-nginx-amazonlinux]
9c8251864520: Pushed
3e8005a3ade1: Pushed
37ab5dcca5fc: Mounted from library/amazonlinux
1.0: digest: sha256:6eb588792d5cf4cd55d54a8b0f1706a3447ff2dea4ab58d81f502146aa362e52 size: 948
[root@ip-172-31-27-111 task4]#
```





4. Conclusion

Successfully completed the exercise of publishing a custom Docker image to Docker Hub.

This task demonstrated how container images can be distributed through a remote registry, allowing developers and operations teams to share and reuse consistent application images across environments.

6. Push image to AWS ECR

1. Objective

The objective of this task was to publish the custom Docker image custom-nginx-amazonlinux:1.0, created in Task 4, to Amazon Elastic Container Registry (ECR).

The image was uploaded to a private ECR repository in the AWS account, making it available for authorized retrieval and deployment.

2. Purpose

The purpose of this exercise was to understand how Docker images are authenticated, tagged, uploaded, and stored in Amazon ECR.

The task covered:

- Configuring IAM permissions for ECR access.
- Creating and attaching an IAM role to an EC2 instance.
- Creating a private ECR repository.
- Authenticating Docker with Amazon ECR.
- Tagging a local Docker image with an ECR repository URI.
- Pushing the image to ECR.
- Verifying the uploaded image and tag.

3. Implementation Steps

Step 1: Verify the Local Docker Image

Executed the following command on the Amazon EC2 instance: sudo docker images

Verified that the custom image was available locally: custom-nginx-amazonlinux:1.0

Step 2: Verify AWS CLI Installation

Executed: aws --version

The AWS CLI was installed successfully. The reported version was: aws-cli/2.36.47

Step 3: Configure IAM Permissions

Created an IAM policy named CustomNginxECRPushPolicy to allow the EC2 instance to authenticate with ECR, manage the required repository, and push or retrieve container images.

The policy included permissions such as:

- ecr:GetAuthorizationToken
- ecr:CreateRepository
- ecr:DescribeRepositories
- ecr:InitiateLayerUpload
- ecr:UploadLayerPart
- ecr:CompleteLayerUpload
- ecr:PutImage
- ecr:DescribeImages

Created an IAM role named EC2CustomNginxECRRole and attached it to the EC2 instance.

Step 4: Verify AWS Identity and Region

Set the AWS region: export AWS_REGION=us-east-1

Verified the AWS identity: aws sts get-caller-identity

The command confirmed that the EC2 instance was using the IAM role EC2CustomNginxECRRole.

AWS account ID: 567487919764
AWS region: us-east-1 (N. Virginia)

Step 5: Create the ECR Repository

Created a private ECR repository named: custom-nginx-amazonlinux

The repository was created successfully in the AWS Management Console.
Repository URI: 567487919764.dkr.ecr.us-east-1.amazonaws.com/custom-nginx-amazonlinux

The repository used AES-256 encryption with the default settings.

Step 6: Authenticate Docker with Amazon ECR

Authenticated Docker using the AWS CLI:

```
aws ecr get-login-password --region "$AWS_REGION" \
  | sudo docker login \
    --username AWS \
    --password-stdin \
    567487919764.dkr.ecr.us-east-1.amazonaws.com
```

The terminal returned: Login Succeeded

This confirmed that Docker authentication with Amazon ECR was successful.

Step 7: Tag the Docker Image

Tagged the existing local image with the ECR repository URI:

```
sudo docker tag \
  custom-nginx-amazonlinux:1.0 \
  567487919764.dkr.ecr.us-east-1.amazonaws.com/custom-nginx-amazonlinux:1.0
```

Verified the image using: sudo docker images

Step 8: Push the Image to Amazon ECR

Executed:

```
sudo docker push \
  567487919764.dkr.ecr.us-east-1.amazonaws.com/custom-nginx-amazonlinux:1.0
```

The image layers were uploaded successfully. The terminal reported that all three listed layers were pushed.

The resulting image digest was:
sha256:6eb588792d5cf4cd55d54a8b0f1706a3447ff2dea4ab58d81f502146aa362e52

Step 9: Verify the Uploaded Image

The successful Docker push confirmed that the image was published to the specified ECR repository with tag 1.0.

The image can also be verified through the AWS Management Console by navigating to:
Amazon ECR → Private repositories → custom-nginx-amazonlinux → Images.
An additional verification command is:

```
aws ecr describe-images \
  --repository-name custom-nginx-amazonlinux \
  --region us-east-1 \
  --query 'imageDetails[*].[imageTags,imageDigest]' \
  --output table
```

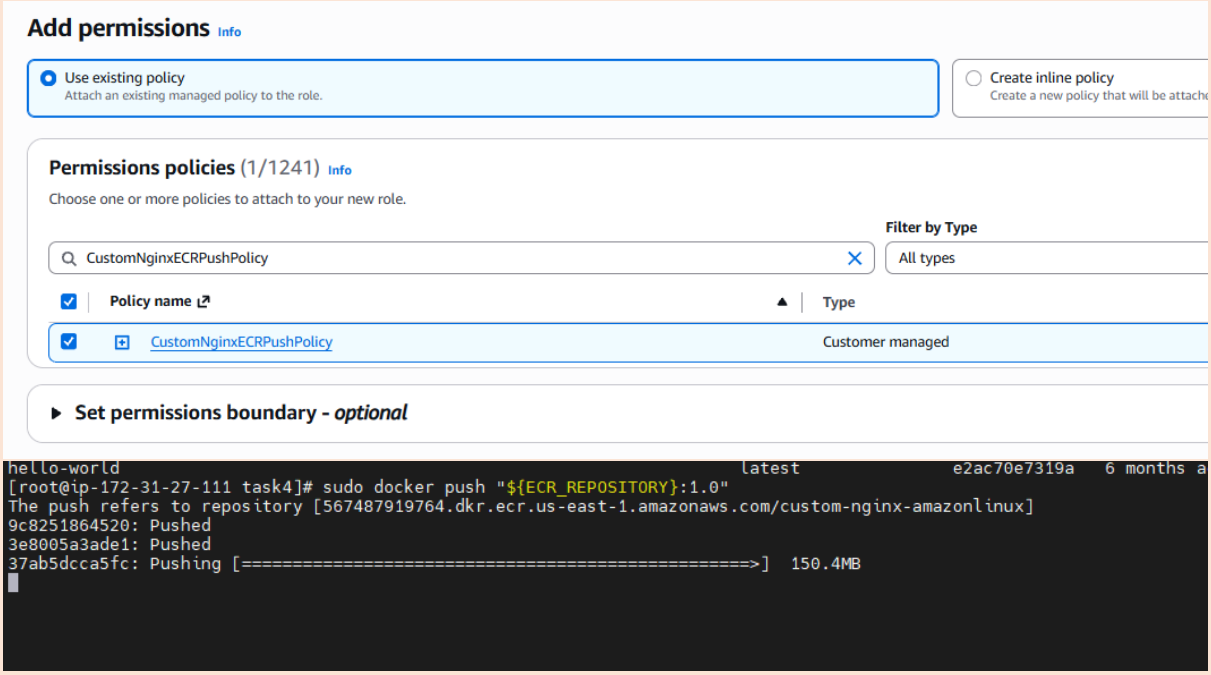
Step 10: Retrieve the Published Image

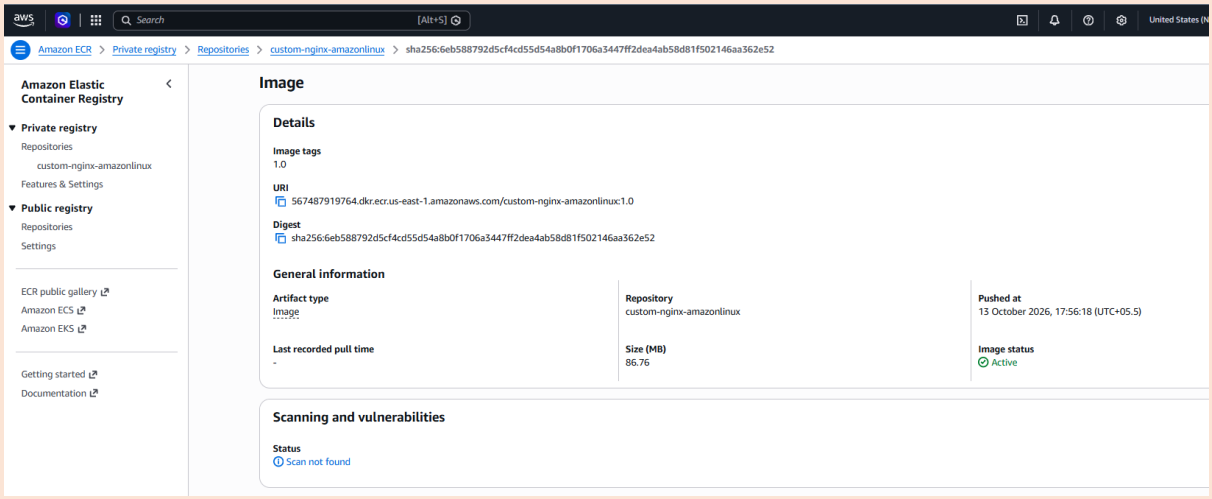
The image can be retrieved from ECR using:

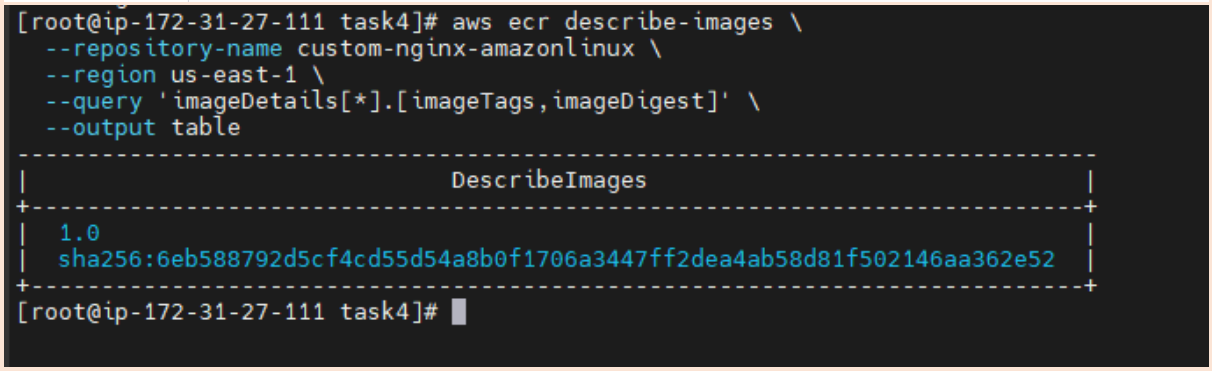
```
sudo docker pull \
  567487919764.dkr.ecr.us-east-1.amazonaws.com/custom-nginx-amazonlinux:1.0
```

This provides an additional way to verify that the published image is available for reuse.

4. Implementation:







5. Conclusion

Successfully pushed the custom Docker image custom-nginx-amazonlinux:1.0 to Amazon Elastic Container Registry.

This task demonstrated how IAM roles and permissions provide secure access to AWS services from an EC2 instance. It also demonstrated how to create a private ECR repository, authenticate Docker, tag a custom image, and upload the image to a remote registry.

The published image is now available in the specified ECR repository for authorized retrieval and future deployment.